RadixGraph: A Fast, Space-Optimized Data Structure for Dynamic Graph Storage

Haoxuan Xie
haoxuan001@e.ntu.edu.sg
Nanyang Technological University, Singapore

Junfeng Liu
junfeng001@e.ntu.edu.sg
Nanyang Technological University, Singapore

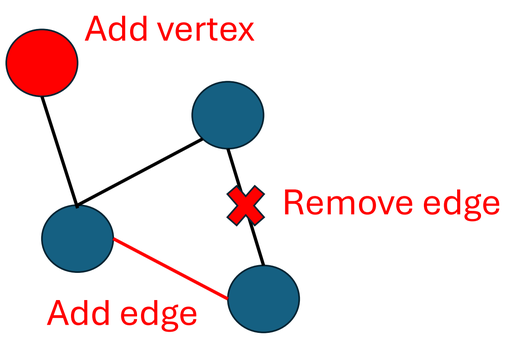
Siqiang Luo
siqiang.luo@ntu.edu.sg
Nanyang Technological University, Singapore

Kai Wang
kai_wang@hit.edu.cn
Harbin Institute of Technology, China

SIGMOD PODS 2026 — BENGALURU, INDIA — May 31-June 5

1. MOTIVATION

- Dynamic graphs are ubiquitous in real-world applications and **evolve continuously**.
- Efficiently maintaining them has become a challenge.



- This requires wise design of both **vertex index** and **edge index**.

- Existing solutions:

Vertex Index

	Update	Query	Space
Static array	Fast	Fast	High
Multi-level vector	Moderate	Fast	Moderate
Tree-based	Moderate	Moderate	Low

Edge Index

	Insert	Delete	Scan
PMA-based	$O(\log^2 d)$	$O(\log^2 d)$	Fast
Log-based	$O(1)$	$O(d)$	Fast
Tree-based	$O(\log d)$	$O(\log d)$	Moderate

"Scan" refers to the efficiency of sequential scan of neighbors. d is the average degree.

2. OUR SOLUTION: RADIXGRAPH

- RadixGraph proposes two innovative designs:

SORT as vertex index

- ✓ Radix-tree based structure to index vertex IDs
- ✓ Among multiple possible tree structures, automatically find the best one
- ✓ Minimizes **average space consumption** under uniformly distributed IDs

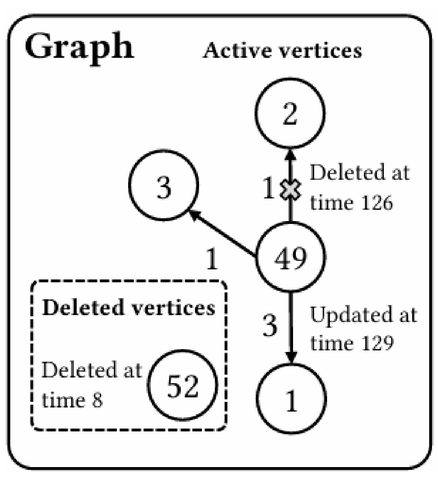
$$T^* = \arg\min_{T \in \mathcal{T}_l} \mathbb{E}_{X \sim \mathcal{U}_d(0, 2^x - 1)} [\text{Space}(T, X, n)]$$

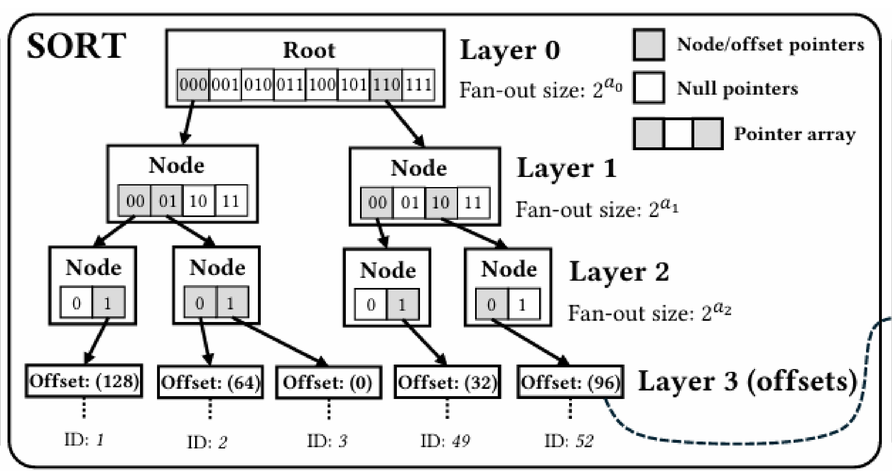
- ✓ Lightweight maintenance overhead (no split/merge operations)
- ✓ Fast query operation and cost independent of how many vertices inserted

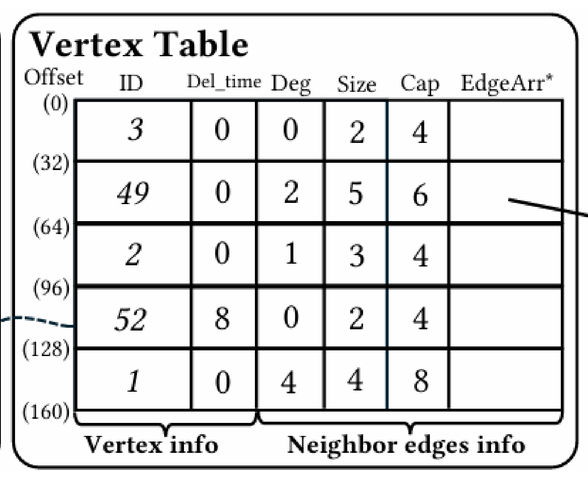
Snapshot-Log edge storage

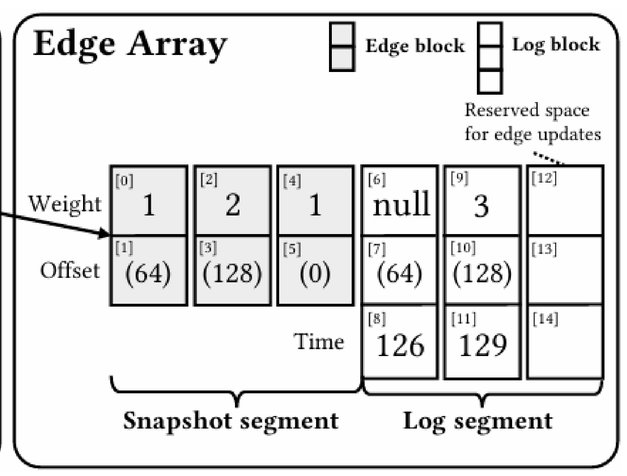
- ✓ Hybrid snapshot-log architecture per vertex
- ✓ Simply an array divided into a snapshot segment and a log segment
- ✓ All updates are firstly logged and then merged into the snapshot segment in batch
- ✓ **Amortized $O(1)$ time** for each edge insertion, update or deletion
- ✓ $O(d)$ time to get neighbor edges and fast sequential scan for graph analytics
- ✓ Space-efficient and cache friendly

Together, they deliver millions of updates per second while keeping queries fast and memory usage low.









(x): an offset corresponding to the location in the vertex table
*In this example, $a_0 = 3, a_1 = 2, a_2 = 1$.

2^{a_i}: pointer-array size of a node at layer i and is pre-determined by our optimizer

3. EXPERIMENTAL RESULTS

Up to 16.27x higher update throughput and average 40.1% less memory usage

RadixGraph is open-source!

https://github.com/ForwardStar/RadixGraph